

1. STRUCTURE DU PROGRAMME

L'architecture de notre application a été conçue pour séparer clairement la logique de traitement (cryptographie) de la logique d'interaction avec l'utilisateur (CLI). Le programme est centralisé autour d'un point d'entrée unique (`main.py`) qui orchestre l'ensemble des opérations.

1.1. Organisation des Modules et Fichiers

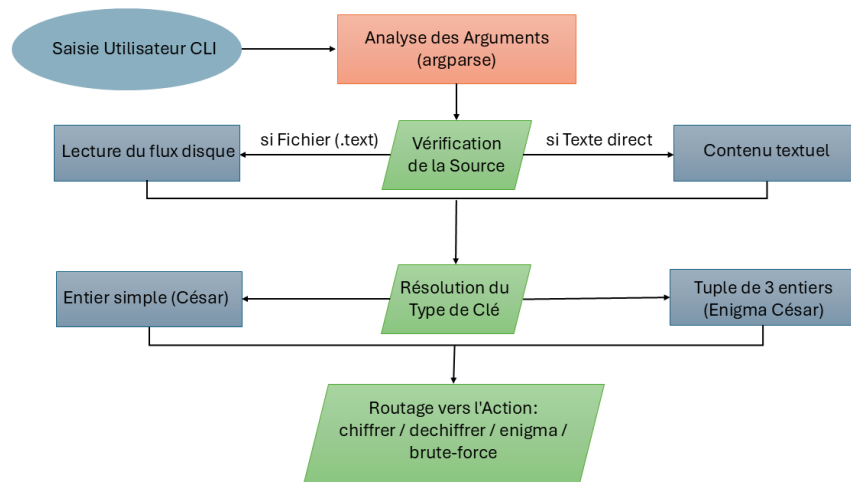
Le projet est structuré de manière modulaire afin de faciliter la maintenance et la lisibilité du code :

- `main.py` : Point d'entrée principal de l'application. Il regroupe les fonctions de haut niveau, l'analyseur d'arguments, la logique de cryptanalyse brute-force, ainsi que les routines d'accès aux fichiers d'entrée/ sortie.
- `tests/test_caesar.py` : Ce module regroupe l'ensemble des tests unitaires exécutés via l'infrastructure `pytest`.
- Fichiers d'entrée (ex. `message.txt`) : Fichier texte standardisé utilisé pour l'évaluation des performances sur des volumes textuels représentatifs.

1.2. Interface en Ligne de Commande (CLI) et Flux de Données

L'interaction avec l'utilisateur est entièrement pilotée par le module `argparse`, assurant une validation stricte des types de paramètres appliqués. Le flux opérationnel suit une séquence déterministe :

Cette structure garantit une validation stricte des paramètres en amont, empêchant les crashes inattendus et offrant des messages de retour clairs à l'utilisateur directement dans le terminal.



Cette structure garantit une validation stricte des paramètres en amont, empêchant les crashes inattendus et offrant des messages de retour clairs à l'utilisateur directement dans le terminal.

2. CONCEPTION ALGORITHMIQUE

Conformément aux directives pédagogiques révisées, l'ensemble du traitement repose sur un principe d'égalisation et de simplification de la casse : toutes les chaînes sont uniformisées en minuscules dès l'amont, éliminant ainsi la complexité inutile liée à la conservation des majuscules dans le flux de sortie.

2.1. Normalisation du Message

La fonction `normaliser_message` neutralise les interférences linguistiques (accents, diacritiques) avant toute manipulation mathématique. Elle applique une décomposition de type NFKD (Normalization Form Compatibility Decomposition) via le module `unicodedata`, permettant d'isoler puis de filtrer les caractères de combinaison, avant de forcer la conversion en minuscules :

```
mot_normalise = unicodedata.normalize('NFKD', message)

message_propre = ''.join([c for c in mot_normalise if not
unicodedata.combining(c)]).lower()
```

2.2. Algorithmes de Chiffrement et Déchiffrement

Chiffrement de César : Repose sur une arithmétique modulaire appliquée à l'index de chaque caractère au sein de l'alphabet ordonné $A = \{0, 1, \dots, 25\}$. La transformation est régie par la relation algébrique suivante :

$$C_i = (P_i + K) \bmod 26$$

Où P_i représente la position de la lettre claire, K la clé entière, et C_i la position résultante. Le déchiffrement implémente l'opération inverse $P_i = (C_i - K) \bmod 26$. L'implémentation Python gère nativement le module des valeurs négatives, garantissant la validité du calcul pour $K < 0$.

Enigma César (Chiffrement Poly alphabétique) : Introduit une dépendance temporelle par l'application cyclique d'un vecteur de clés $K = (k_0, k_1, k_2)$. Un pointeur d'indexation j , synchronisé uniquement sur les caractères alphabétiques traités, évolue selon la règle récursive :

$$j_{next} = (j + 1) \bmod 3$$

2.3. Cryptanalyse Brute-Force et Filtrage

L'attaque par force brute consiste à explorer l'espace des clés de manière exhaustive. Pour César, la dimension spatiale est restreinte ($N = 26$). Pour Enigma César, l'espace croît de

façon combinatoire à $N = 26^3 = 17576$ combinaisons. Les permutations sont générées via l'itérateur hautement optimisé `itertools.product(range(26), repeat=3)`, écrit en C sous le capot, minimisant le surcoût lié à l'interprétation de la boucle.

Heuristique de Détection Automatique (Filtrage) : Pour identifier la bonne clé sans intervention humaine, la fonction `est_francais` analyse la pertinence sémantique du texte partiel produit. Elle extrait les chaînes de caractères pures à l'aide d'expressions régulières (`re.findall(r' [a-z]+ ', texte)`) et comptabilise les correspondances avec un dictionnaire de mots fonctionnels francophones hautement fréquents (le, la, les, et, de, du, un, une, est, que, dans, pour).

3. ÉVALUATION DES PERFORMANCES

Grâce aux structures en $O(1)$ et aux itérateurs natifs, les temps d'exécution (mesurés avec `perf_counter()`) sont extrêmement faibles, même avec l'expansion exponentielle d'Enigma :

Algorithme d'Attaque	Espace des Clés	Complexité Théorique	Statut de Validation
Brute-Force César	26	$O(26 \times M)$	Succès
Brute-Force Enigma	17576	$O(26^3 \times M)$	Succès (Seuil ≥ 3)

4. DISTRIBUTION DES TÂCHES

Le travail a été équitablement réparti pour maximiser l'efficacité :

Membre de l'Équipe	Responsabilités Techniques et Algorithmiques Réalisées
Nicolas Allard	- Développement des algorithmes de base pour le chiffrement (César et Enigma) - Mise en place de la normalisation du texte (NFKD) et conception de la suite de tests unitaires (<code>pytest</code>) pour garantir la fiabilité des transformations.
Hamza Amri-Jouidel	- Conception et codage des algorithmes de déchiffrement (César et Enigma). - Implémentation de l'arithmétique modulaire inverse et de la gestion du cycle des clés pour le déchiffrement poly alphabétique.
Mohammad Shohoudi mojdehi	- Développement du code de cryptanalyse (Brute-Force) et de l'heuristique d'analyse linguistique - Intégration globale du programme via l'interface en ligne de commande (CLI avec <code>argparse</code>), incluant la gestion des flux de fichiers et la mesure des performances.